A DEEP LEARNING MINI-PROJECT

on

Netra.AI

Detection of Retinal Diseases from Fundus Images using VGG19 Transfer Learning

Submitted towards the
Fulfilment of the Requirements of

Bachelor of Technology in Computer Engineering

By

Name	Roll No.	Seat No.
Hrishikesh Gavai	17	B231301040
Deepak Warude	18	B231301148
Atharva Ghayal	19	B231301042
Tejas Sangle	20	B231301122

Under the guidance of

Prof. Shweta Jadhav

Department of Computer Engineering
K. K. WAGH EDUCATION SOCIETY'S
K. K. WAGH INSTITUTE OF ENGINEERING EDUCATION AND RESEARCH
Hirabai Haridas Vidyanagari, Amrutdham, Panchavati, Nashik, Maharashtra 422003
(An Autonomous Institute affiliated to Savitribai Phule Pune University)

A. Y. 2026-2027 Sem VII

Abstract

Cataract, diabetic retinopathy and glaucoma are some of the most common causes of vision loss, and all three can be seen on a photograph of the back of the eye, called a fundus image. Reading these images needs an eye specialist, and there are not enough specialists for the number of patients who need screening.

Netra.AI is a web application that uses deep learning to classify a fundus image into one of four classes: Cataract, Diabetic Retinopathy, Glaucoma or Normal. The model is based on VGG19, a convolutional neural network pre-trained on the ImageNet dataset. We used transfer learning: the convolutional layers of VGG19 were kept frozen and only a small classification head (Flatten, Dense 128, Dropout 0.5, Dense 4) was trained on our data. The dataset has 4,217 images in two sizes (256×256 and 512×512). All images are resized to 224×224 before training.

On the validation set of 841 images, the model achieved an overall accuracy of 73.96%. It detects diabetic retinopathy very well (F1-score 98.21%) and cataract well (F1-score 83.20%), but it often confuses glaucoma with normal eyes. After the prediction, the application uses the Google Gemini 3.5 Flash API to explain the result in simple language and a web scraping module to list nearby eye hospitals. The application is hosted on Google Cloud Run.

Keywords: Deep Learning, CNN, VGG19, Transfer Learning, Fundus Image, Diabetic Retinopathy, Glaucoma, Cataract, Gemini API, Cloud Run.

Contents

Abstract **i**

1 Introduction **1**
1.1 Background 1
1.2 Motivation 1
1.3 Objectives 1
1.4 Scope 2

2 Problem Statement **3**
2.1 Challenges 3

3 Literature Survey **4**

4 Dataset **5**
4.1 Dataset Details 5
4.2 Features of Each Class 6
4.3 Training and Validation Split 6

5 Data Pre-processing **7**
5.1 Resizing 256×256 and 512×512 Images to 224×224 7
5.2 Why 224×224? 8
5.3 Normalization 8

6 Model Architecture **9**
6.1 Convolutional Neural Network (CNN) 9
6.2 Why VGG19? 9
6.3 VGG19 Architecture 10
6.4 Layer-by-Layer Working 12
6.5 Transfer Learning 12

7 Model Training **13**
7.1 Training Pipeline 13
7.2 Training Parameters 14
7.3 Loss Function and Optimizer 14
7.4 Training Steps 14

8 System Implementation **16**
8.1 System Architecture 16
8.2 Prediction in the Web Application 16
8.3 Gemini 3.5 Flash API 17
8.4 Doctor Finder (Web Scraping) 17
8.5 Technologies Used 18
8.6 Hosting 18

9 Results **19**
9.1 Evaluation Metrics 19
9.2 Overall Performance 19
9.3 Class-wise Performance 20
9.4 Confusion Matrix 20

10 Observations **22**
10.1 Classes That Worked Well 22
10.2 Classes That Did Not Work Well 22
10.3 Why the Final Accuracy Is Lower 22
10.4 Summary 23

11 Conclusion and Future Scope **24**
11.1 Conclusion 24
11.2 Future Scope 25

References **26**

Appendix A Project Structure **28**

Disclaimer **29**

List of Figures

4.1 Sample images from each class 5

5.1 Resizing images of two sizes to 224×224 8

6.1 Example of convolution, ReLU and max pooling 9
6.2 VGG19 architecture with the custom classification head 11

7.1 Model training pipeline 13

8.1 System architecture of Netra.AI 16

9.1 Confusion matrix heatmap 20
9.2 Class-wise metrics and overall performance 21

List of Tables

2.1 Challenges and how they are handled 3

3.1 Summary of related work 4

4.1 Dataset specifications 5
4.2 Visible features of each class 6
4.3 Training and validation split 6

5.1 Effect of resizing 7

6.1 Comparison of popular pre-trained models (values from Keras Applications [10]) 10
6.2 Layer-wise details of the model 11

7.1 Training parameters 14

8.1 Data sources of the doctor finder 17
8.2 Technologies used 18

9.1 Overall performance 19
9.2 Precision, recall and F1-score for each class 20
9.3 Confusion matrix (rows = actual class, columns = predicted class) 20
9.4 TP, FP, FN and TN for each class 21

10.1 Summary of class-wise results 23

11.1 Future scope 25

Chapter 1

Introduction

1.1 Background

Eye problems affect a very large part of the world's population. The WHO report on vision estimates the number of people with some form of visual impairment at more than 2.2 billion, and roughly half of these cases are ones that could have been avoided or are still untreated [1]. Cataract remains the biggest single cause of blindness, the number of diabetic retinopathy (DR) cases keeps growing with the number of diabetics, and glaucoma patients are projected to cross 111 million by the year 2040 [2].

All three of these diseases show visible signs on a fundus image (a photograph of the inner back surface of the eye). Taking such a photograph has become cheap and quick, but someone still has to read it, and that needs an ophthalmologist. Smaller towns often do not have one nearby, so many patients reach a doctor only when the damage to their vision is already done.

1.2 Motivation

Deep learning models, especially Convolutional Neural Networks (CNNs), have shown very good results in medical image classification. Studies such as Gulshan et al. [3] showed that a CNN can detect diabetic retinopathy from fundus images as accurately as eye doctors. This motivated us to build a smaller version of such a system as our Deep Learning mini-project. The project also lets us apply the main topics of the subject (CNNs, transfer learning, activation functions, loss functions, optimizers, dropout and model evaluation) on a real problem.

We also wanted the system to be useful for a normal user. A prediction alone does not help much, so the application explains the result in simple words and helps the user find a nearby eye specialist.

1.3 Objectives

1. To build a CNN model that classifies fundus images into Cataract, Diabetic Retinopathy, Glaucoma and Normal.
2. To use transfer learning with the pre-trained VGG19 model, since our dataset is small.
3. To pre-process images of different sizes into the 224×224 input required by VGG19.

4. To evaluate the model using accuracy, precision, recall, F1-score and the confusion matrix.
5. To give a simple explanation of the result using the Gemini API and show nearby eye hospitals.
6. To deploy the complete application on Google Cloud Run.

1.4 Scope

Netra.AI is an educational prototype and not a certified medical device. It is meant to assist doctors and users, not to replace a proper eye examination. The model gives one label per image, so it cannot report two diseases in the same eye.

Chapter 2

Problem Statement

"To develop a deep learning based system that takes a colour fundus image as input, classifies it as Cataract, Diabetic Retinopathy, Glaucoma or Normal using a VGG19 transfer learning model, and gives the user an easy explanation of the result along with nearby eye specialist details."

2.1 Challenges

Table 2.1: Challenges and how they are handled

Challenge	**How Netra.AI handles it**
Disease signs are small and look similar to each other	A deep CNN (VGG19) learns these features automatically
Dataset is small (about 4,200 images)	Transfer learning with frozen VGG19 layers
Images come in two different sizes	All images are resized to 224×224
Model may overfit the training data	Dropout layer and a separate validation set
Users cannot understand model output	Gemini API gives a simple explanation
User does not know where to go next	Doctor finder shows nearby eye hospitals

Chapter 3

Literature Survey

CNNs became popular after AlexNet [5] won the ImageNet competition in 2012. Simonyan and Zisserman [6] then proposed VGG16 and VGG19, which use only small 3×3 filters but stack many layers to make the network deep. ResNet [7] later added skip connections so that even deeper networks could be trained.

In eye disease detection, Gulshan et al. [3] trained a CNN on about 1.28 lakh fundus images for diabetic retinopathy and reached accuracy similar to eye specialists. Ting et al. [4] built a system for DR, glaucoma and other eye diseases and tested it on patients from different countries. Yosinski et al. [8] showed that the first layers of a CNN learn general features like edges and colours, which is why a model trained on ImageNet can be reused for other image tasks such as medical images.

Table 3.1: Summary of related work

Paper	**Method**	**Finding**
Krizhevsky et al., 2012 [5]	AlexNet	Deep CNNs work far better than older methods on ImageNet
Simonyan & Zisserman, 2015 [6]	VGG16 / VGG19	Deeper networks with 3×3 filters give better accuracy
He et al., 2016 [7]	ResNet	Skip connections allow very deep networks
Yosinski et al., 2014 [8]	Transfer learning study	Early CNN layers are general and can be reused
Gulshan et al., 2016 [3]	Inception-v3	DR detection at specialist level on a very large dataset
Ting et al., 2017 [4]	CNN ensemble	Detection of DR, glaucoma and other eye diseases

Most of these systems use very large private datasets. Our project uses a small public dataset, a simple and well known model (VGG19), and adds an explanation and doctor finder for the user.

Chapter 4

Dataset

4.1 Dataset Details

We used the public "Eye Diseases Classification" dataset from Kaggle [11]. It contains colour fundus images in four folders, one for each class. The number of images and their original sizes are given in Table 4.1.

Table 4.1: Dataset specifications

Class	**No. of Images**	**Image Size**
Cataract	1038	256 × 256
Diabetic Retinopathy	1098	512 × 512
Glaucoma	1007	256 × 256
Normal	1074	512 × 512
Total	**4217**	–

The classes are almost balanced: the largest class (DR) has only about 9% more images than the smallest class (Glaucoma). All images are RGB colour images in JPG or PNG format.

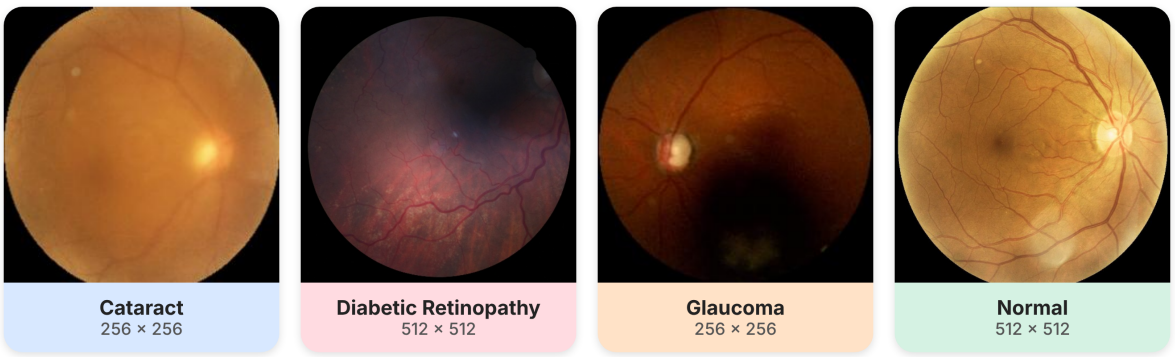


Figure 4.1: Sample images from each class

4.2 Features of Each Class

Table 4.2: Visible features of each class

Class	**Features seen in the fundus image**
Cataract	Whole image looks hazy and blurred, blood vessels are not clear
Diabetic Retinopathy	Small red spots (haemorrhages), yellow spots (exudates), abnormal blood vessels
Glaucoma	Larger and paler cup inside the optic disc (high cup-to-disc ratio)
Normal	Clear blood vessels, sharp optic disc and darker macula

4.3 Training and Validation Split

The data is split using `ImageDataGenerator(validation_split=0.2)`. This splits every class folder separately, so 80% of each class is used for training and 20% for validation.

Table 4.3: Training and validation split

Class	**Total**	**Training**	**Validation**
Cataract	1038	831	207
Diabetic Retinopathy	1098	879	219
Glaucoma	1007	806	201
Normal	1074	860	214
Total	**4217**	**3376**	**841**

Chapter 5

Data Pre-processing

Before training, every image goes through the same steps. The same steps are also used in the web application when a user uploads an image, so the model always gets input in the same form.

1. **Resize:** the image is resized to 224×224 pixels.
2. **Convert:** the image is converted to an array of RGB pixel values.
3. **Normalize:** each pixel value is divided by 255 so that it lies between 0 and 1.
4. **Batch:** images are grouped into batches of 32 for training. For a single upload the input shape becomes (1, 224, 224, 3).

5.1 Resizing 256×256 and 512×512 Images to 224×224

Our dataset has images of two sizes: Cataract and Glaucoma images are 256×256, while DR and Normal images are 512×512. A neural network needs all inputs to be the same size, so all images are resized to 224×224. This is done automatically by Keras:

- During training, `flow_from_directory(target_size=(224, 224))` resizes each image while loading it.
- During prediction, `image.load_img(path, target_size=(224, 224))` does the same for the uploaded image.

Keras uses **nearest-neighbour** interpolation by default. Each pixel in the new 224×224 image is copied from the closest pixel in the original image, so no new colours are created. Since all images are square, the image is not stretched or cropped and the full eye stays visible.

Table 5.1: Effect of resizing

Original Size	Classes	Scale Factor	Effect
256 × 256	Cataract, Glaucoma	256 / 224 = 1.14	Small reduction, most detail is kept
512 × 512	DR, Normal	512 / 224 = 2.29	Large reduction, some fine detail is lost

5.2 Why 224×224?

- VGG19 was originally trained on ImageNet images of size 224×224, so its learned filters work best at this size.
- VGG19 has 5 pooling layers, and each one halves the image size: 224 → 112 → 56 → 28 → 14 → 7. This gives a final feature map of 7×7×512, which is what our Flatten and Dense layers are built for.
- It keeps the memory and computation small enough for training and for cloud deployment.

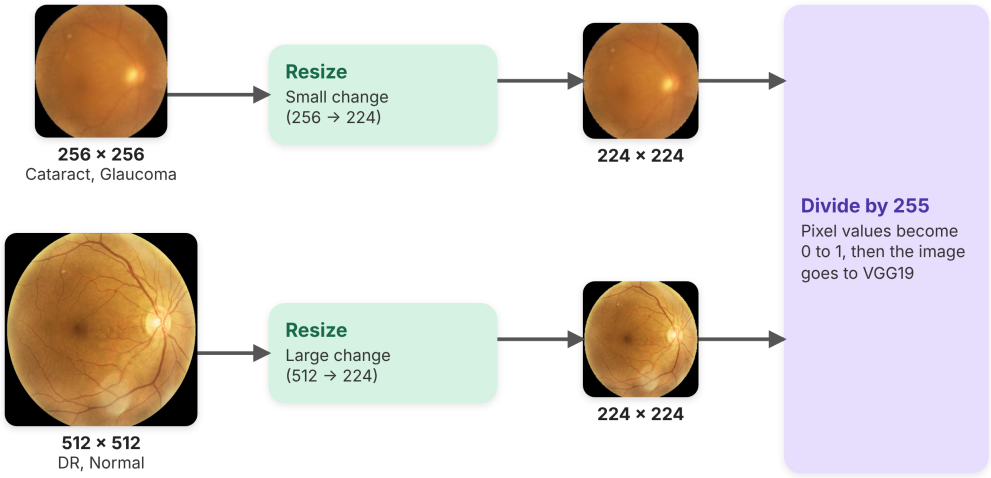


Figure 5.1: Resizing images of two sizes to 224×224

5.3 Normalization

Pixel values are originally between 0 and 255. They are divided by 255 using `rescale=1./255`, which brings them between 0 and 1. Small input values make training more stable. No data augmentation (flipping, rotation, etc.) was used in this project.

Chapter 6

Model Architecture

6.1 Convolutional Neural Network (CNN)

A CNN is a type of neural network designed for images. Instead of looking at the whole image at once, it slides small filters over the image to find patterns. The main layers are:

- **Convolution layer:** a small filter (3×3 in VGG19) moves over the image and produces a feature map. Each filter learns to detect one pattern, such as an edge or a colour change.
- **ReLU activation:** replaces negative values with zero, i.e. $\mathrm{ReLU}(x) = \max(0, x)$. This adds non-linearity so the network can learn complex patterns.
- **Max pooling:** takes the largest value from each 2×2 block, which halves the size of the feature map and keeps the strongest features.
- **Fully connected (Dense) layer:** uses all the extracted features to make the final decision.
- **Softmax:** converts the final outputs into probabilities that add up to 1.

Figure 6.1 shows a small example of convolution, ReLU and max pooling with real numbers.

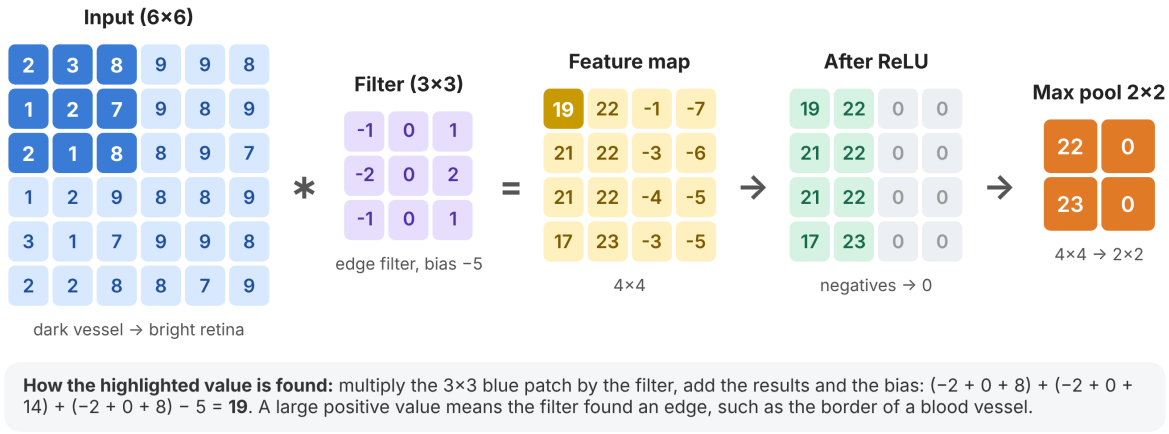


Figure 6.1: Example of convolution, ReLU and max pooling

6.2 Why VGG19?

We chose VGG19 for the following reasons:

1. **Simple structure:** it uses only 3×3 convolutions and 2×2 pooling, repeated in 5 blocks. This makes it easy to understand and explain layer by layer.
2. **Good pre-trained features:** it is trained on more than 1 million ImageNet images, so its layers already detect edges, textures and shapes that are also useful for fundus images.
3. **Works well with small datasets:** our dataset has only 4,217 images, which is too small to train a deep network from scratch. With transfer learning we only train a small part of the model.
4. **Deeper than VGG16:** blocks 3, 4 and 5 have 4 convolution layers instead of 3, so the model can learn slightly more detailed features.
5. **Easy to use:** it is available directly in Keras with pre-trained weights.

Table 6.1: Comparison of popular pre-trained models (values from Keras Applications [10])

Model	Size (MB)	Parameters	Top-1 Acc.	Depth
VGG16	528	138.4 M	71.3%	16
VGG19 (used)	**549**	**143.7 M**	**71.3%**	**19**
ResNet50	98	25.6 M	74.9%	107
InceptionV3	92	23.9 M	77.9%	189
MobileNetV2	14	3.5 M	71.3%	105
EfficientNetB0	29	5.3 M	77.1%	132

VGG19 is heavier than newer models like MobileNet or EfficientNet. However, most of its parameters are in its original fully connected layers, which we do not use (`include_top=False`). The part we use has about 20 million parameters.

6.3 VGG19 Architecture

VGG19 has 16 convolution layers and 3 fully connected layers (19 layers with weights). In our model, the original fully connected layers are removed and replaced with our own classification head. Figure 6.2 shows how an image moves through the network.

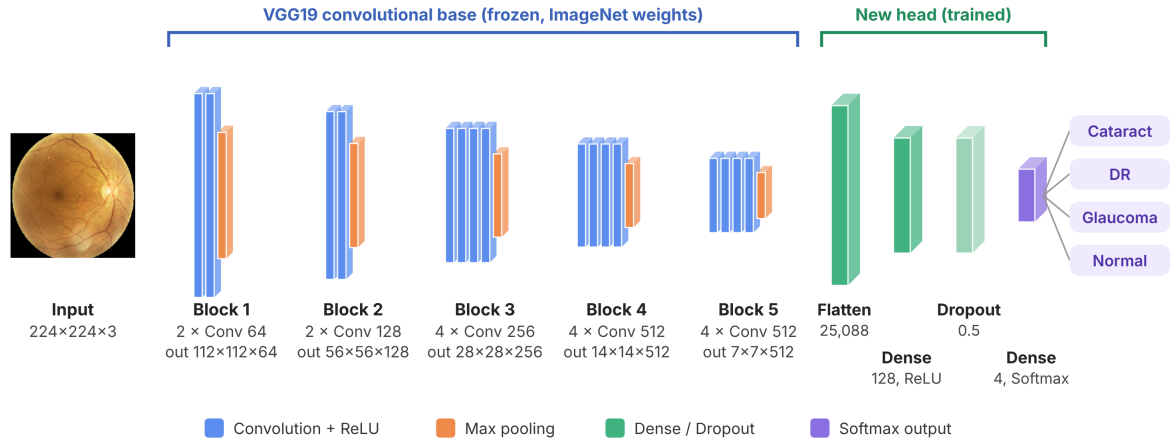


Figure 6.2: VGG19 architecture with the custom classification head

Table 6.2: Layer-wise details of the model

Block	**Layers**	**Output Size**	**Parameters**
Input	RGB image	224×224×3	0
Block 1	2 Conv (64 filters) + MaxPool	112×112×64	38,720
Block 2	2 Conv (128 filters) + MaxPool	56×56×128	221,440
Block 3	4 Conv (256 filters) + MaxPool	28×28×256	2,065,408
Block 4	4 Conv (512 filters) + MaxPool	14×14×512	8,259,584
Block 5	4 Conv (512 filters) + MaxPool	7×7×512	9,439,232
Flatten	Converts to 1-D vector	25,088	0
Dense	128 neurons, ReLU	128	3,211,392
Dropout	Rate 0.5	128	0
Dense (Output)	4 neurons, Softmax	4	516
Total parameters			**23,236,292**
Frozen (VGG19 layers, not trained)			20,024,384
Trainable (our head)			3,211,908

6.4 Layer-by-Layer Working

- **Block 1:** finds simple features like edges, borders of blood vessels and colour changes.
- **Block 2:** combines edges into small textures and thin vessel lines; small red and yellow spots start to get detected here.
- **Block 3:** finds larger patterns such as vessel branches, groups of spots and the hazy look of cataract.
- **Block 4:** recognises bigger structures like the optic disc and areas with many lesions.
- **Block 5:** each output now covers almost the whole image and describes the overall appearance of the eye.
- **Flatten and Dense layers:** the 7×7×512 feature map is converted to 25,088 values. The Dense layer with 128 neurons learns which features belong to which disease, Dropout randomly turns off 50% of these neurons during training to reduce overfitting, and the last Dense layer with Softmax gives the probability of each class.

6.5 Transfer Learning

Transfer learning means using a model that was already trained on one task for a new task. We loaded VGG19 with ImageNet weights and set `layer.trainable = False` for all its layers. So the VGG19 layers act only as a feature extractor, and only our new head (3.2 million parameters) is trained on the fundus images.

```
base_model = VGG19(weights="imagenet", include_top=False, input_shape=(224, 224, 3))
for layer in base_model.layers:
    layer.trainable = False  # Freeze pretrained layers

x = Flatten()(base_model.output)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(len(CATEGORIES), activation='softmax')(x)
model = Model(inputs=base_model.input, outputs=output)
```

Listing 6.1: Building the model (model_training.py)

Chapter 7

Model Training

7.1 Training Pipeline

Figure 7.1 shows the complete training process, from loading the dataset to saving the trained model.

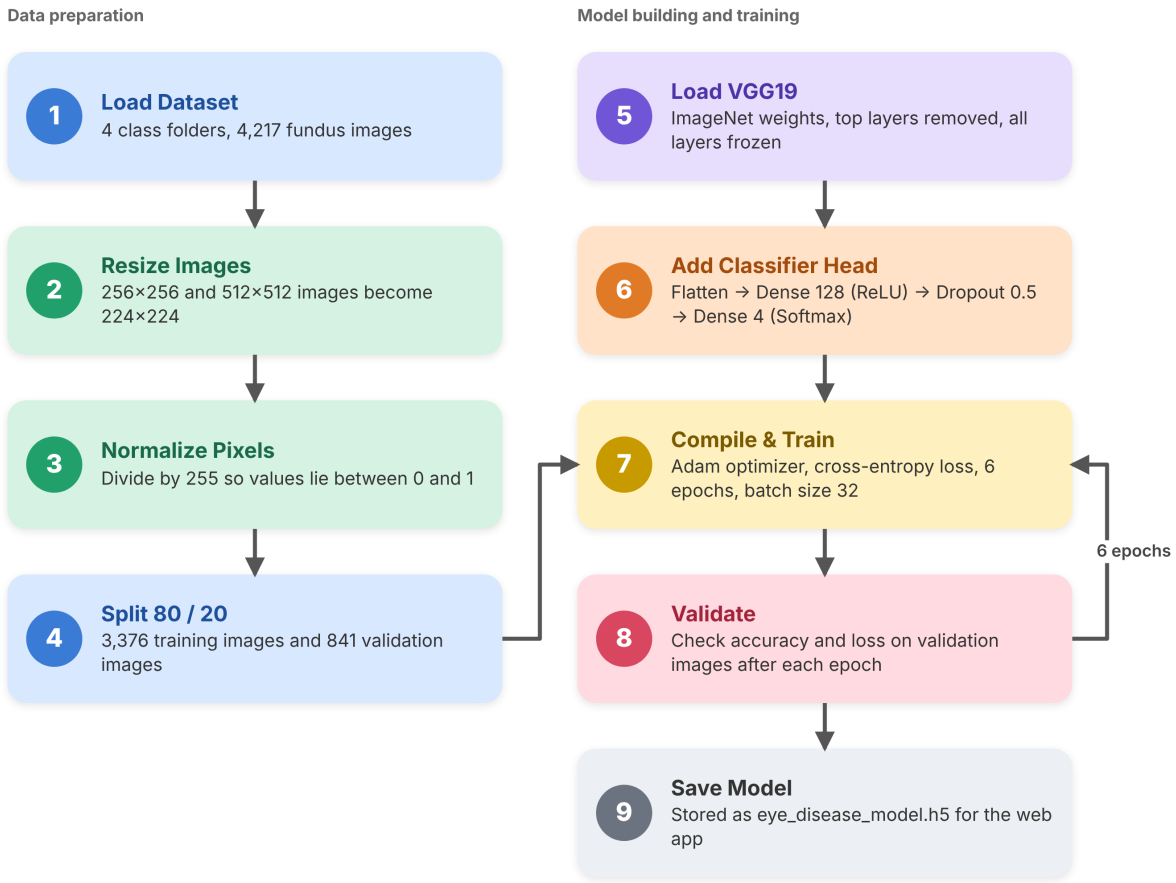


Figure 7.1: Model training pipeline

7.2 Training Parameters

Table 7.1: Training parameters

Parameter	**Value**
Base model	VGG19 (ImageNet weights, frozen)
Input size	$224 \times 224 \times 3$
Optimizer	Adam (learning rate 0.001)
Loss function	Categorical Cross-Entropy
Epochs	6
Batch size	32
Steps per epoch	106
Train / Validation split	80% / 20%
Data augmentation	Not used
Saved model	models/eye_disease_model.h5

7.3 Loss Function and Optimizer

Categorical Cross-Entropy is used because this is a multi-class problem with one-hot labels. The loss is small when the model gives a high probability to the correct class and large when it does not:

$$\text{Loss} = -\log(p_{\text{correct class}}) \tag{7.1}$$

Adam is used as the optimizer. It updates each weight using the average of recent gradients and adjusts the step size for each weight automatically, so it usually trains faster and more smoothly than plain gradient descent. We used the default learning rate of 0.001.

7.4 Training Steps

1. Load images from the dataset folders with `ImageDataGenerator` (rescale and 80/20 split).
2. Load VGG19 without its top layers and freeze all its layers.
3. Add the head: Flatten, Dense(128, ReLU), Dropout(0.5), Dense(4, Softmax).
4. Compile the model with the Adam optimizer and categorical cross-entropy loss.
5. Train for 6 epochs with a batch size of 32 and check validation accuracy after every epoch.

6. Save the trained model as `eye_disease_model.h5`.

```
datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)
train_data = datagen.flow_from_directory(DATASET_PATH, target_size=IMG_SIZE,
    batch_size=BATCH_SIZE, class_mode='categorical', subset="training")
val_data = datagen.flow_from_directory(DATASET_PATH, target_size=IMG_SIZE,
    batch_size=BATCH_SIZE, class_mode='categorical', subset="validation")

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'
    ])
history = model.fit(train_data, validation_data=val_data, epochs=EPOCHS)
model.save("models/eye_disease_model.h5")
```

Listing 7.1: Data loading and training (model_training.py)

Chapter 8

System Implementation

8.1 System Architecture

The trained model is used inside a small web application. Figure 8.1 shows how the parts are connected. The user uploads a fundus photo from the browser. The Flask server prepares the image in the same way as during training and passes it to the VGG19 model. The predicted disease and its confidence are then sent to the Gemini API, which returns a short explanation. Both are shown on the result page, from where the user can open the doctor finder.

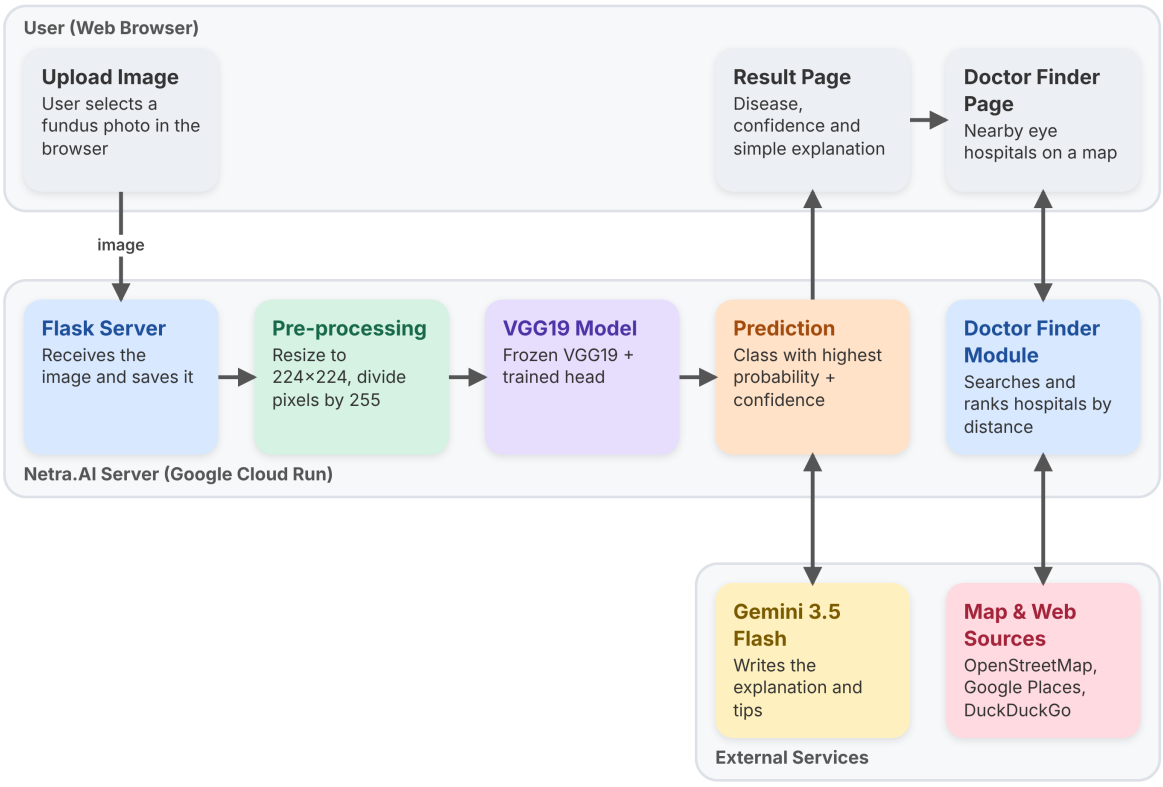


Figure 8.1: System architecture of Netra.AI

8.2 Prediction in the Web Application

When the server starts, it loads the saved model file `eye_disease_model.h5` into memory once. For every uploaded image the server:

1. resizes the image to 224×224 and divides the pixel values by 255,
2. adds a batch dimension so the input shape becomes (1, 224, 224, 3),

3. runs the model, which returns four probabilities (one per class),
4. picks the class with the highest probability as the answer and shows that probability as the confidence in percent.

The probabilities of all four classes are also displayed as bars, so the user can see how sure the model is.

8.3 Gemini 3.5 Flash API

A label such as "Glaucoma, 81%" does not tell a normal person much. So after the prediction, the disease name and confidence are passed to Google's Gemini 3.5 Flash model through the `google-genai` Python library. Gemini 3.5 Flash is a fast language model of the Gemini 3 family released in 2026 [13, 12]. We ask it to answer in a fixed JSON format with five parts: a short description of the disease, common symptoms, risk factors, next steps and lifestyle tips. These are shown on the result page.

If the API is busy, the request is retried a few times. If it still fails, a ready-made text stored in the code is shown instead, so the user always gets some explanation.

8.4 Doctor Finder (Web Scraping)

The doctor finder (`web_scraper.py`) takes a city name and returns up to 15 eye hospitals near it. It first finds the location of the city and then collects hospitals from the sources in Table 8.1.

Table 8.1: Data sources of the doctor finder

Source	**Use**
City coordinate list	Stored latitude and longitude of 54 Indian cities
Nominatim (OpenStreetMap)	Location of cities that are not in the list
Google Places API	Hospitals with ratings and phone numbers (only if a key is given)
Overpass API (OpenStreetMap)	Eye hospitals and clinics within 25 km [15]
Curated list	Checked eye hospitals for 14 large cities
DuckDuckGo search	Official website link of a hospital

Repeated hospitals are removed, the distance of each one from the city centre is worked out from the coordinates, and the list is sorted with the closest hospitals first. The results are shown as cards and as markers on a map.

8.5 Technologies Used

Table 8.2: Technologies used

Technology	**Purpose**
Python 3.10	Main programming language
TensorFlow / Keras 2.15	Building, training and running the model
NumPy, scikit-learn	Array handling and evaluation metrics
Matplotlib, Seaborn	Graphs and confusion matrix
Flask	Web server
Google Gemini 3.5 Flash	Explanation of the result
Requests, BeautifulSoup	Web scraping for the doctor finder
HTML, CSS, JavaScript, Leaflet.js	User interface and map
Google Cloud Run	Hosting of the application

8.6 Hosting

The whole application, together with the trained model, is packed into a Docker container and hosted on Google Cloud Run [14], so it can be opened from any browser through a public link.

Chapter 9

Results

9.1 Evaluation Metrics

The model was evaluated on the 841 validation images using `evaluate_model.py`. The following terms are used for each class:

- **TP (True Positive):** images of the class that were predicted correctly.
- **FP (False Positive):** images of other classes wrongly predicted as this class.
- **FN (False Negative):** images of this class predicted as some other class.
- **TN (True Negative):** images of other classes correctly not predicted as this class.

The metrics are calculated using the following formulae:

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}} \tag{9.1}$$

$$\text{Error Rate} = 1 - \text{Accuracy} \tag{9.2}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{9.3}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{9.4}$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{9.5}$$

Precision tells us how many of the predicted cases of a class were correct. Recall tells us how many of the actual cases of a class were found by the model. F1-score combines both into one value.

9.2 Overall Performance

Table 9.1: Overall performance

Metric	**Value**
Accuracy	**73.96%**
Error Rate	26.04%
Weighted Precision	77.40%
Weighted Recall	73.96%
Weighted F1-Score	72.39%

The model predicted 622 out of 841 validation images correctly.

9.3 Class-wise Performance

Table 9.2: Precision, recall and F1-score for each class

Class	Precision (%)	Recall (%)	F1-Score (%)
Cataract	92.86	75.36	83.20
Diabetic Retinopathy	96.48	100.00	98.21
Glaucoma	67.06	28.36	39.86
Normal	52.63	88.79	66.09

9.4 Confusion Matrix

Table 9.3: Confusion matrix (rows = actual class, columns = predicted class)

	Cataract	DR	Glaucoma	Normal
Cataract	**156**	2	11	38
DR	0	**219**	0	0
Glaucoma	5	6	**57**	133
Normal	7	0	17	**190**

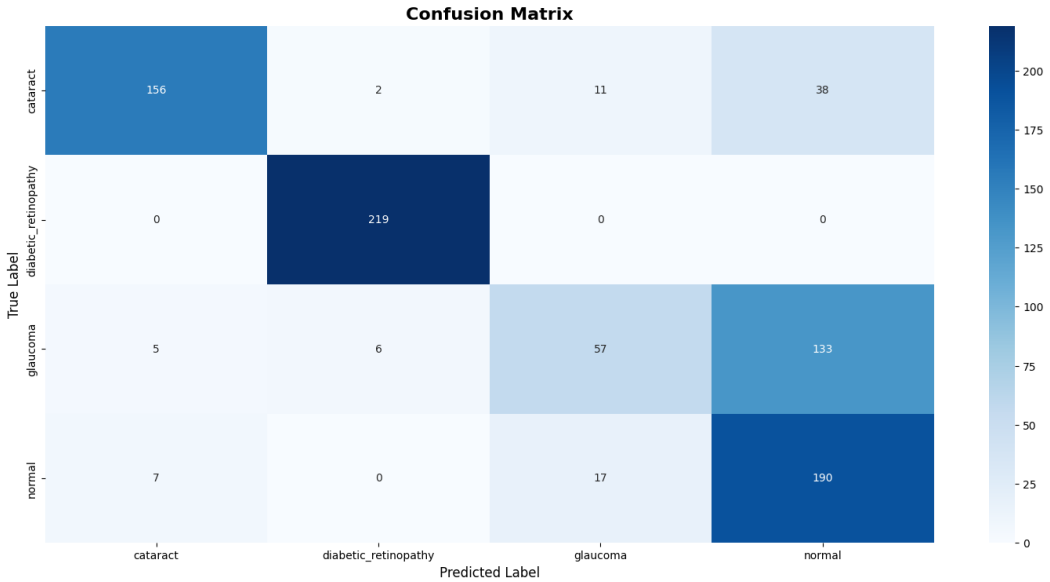


Figure 9.1: Confusion matrix heatmap

Table 9.4: TP, FP, FN and TN for each class

Class	TP	FP	FN	TN
Cataract	156	12	51	622
Diabetic Retinopathy	219	8	0	614
Glaucoma	57	28	144	612
Normal	190	171	24	456

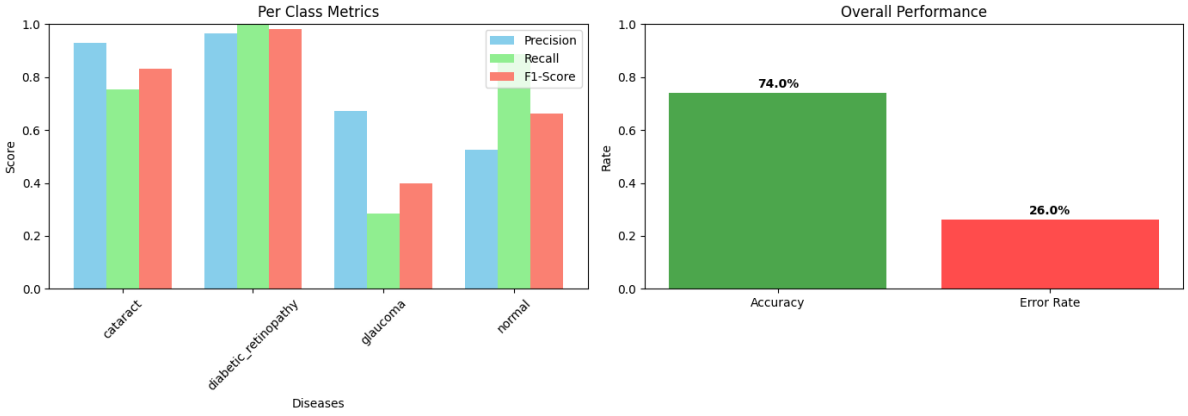


Figure 9.2: Class-wise metrics and overall performance

Chapter 10

Observations

10.1 Classes That Worked Well

Diabetic Retinopathy has the best score (F1-score 98.21%). Every one of the 219 DR images in the validation set was identified. DR leaves many red and yellow marks across the retina, and these textures are easy for the VGG19 filters to respond to.

Cataract is also handled well (F1-score 83.20%). Its precision of 92.86% means the model is rarely wrong when it says "cataract". The 38 cataract images marked as Normal are probably early cases where the haze is very faint.

10.2 Classes That Did Not Work Well

Glaucoma is the weakest class, with a recall of only 28.36%. Of the 201 glaucoma images, 133 were labelled Normal. This is the most serious mistake the model makes, because a glaucoma patient would be told that the eye is fine.

Normal shows the other side of the same problem. Its recall is high (88.79%) but its precision is low (52.63%): the model said "Normal" 361 times when only 214 images were really normal, and most of the extra ones were glaucoma images.

10.3 Why the Final Accuracy Is Lower

An accuracy of 73.96% is far better than random guessing (25% for four classes), but it is lower than we would like. The main reasons are given below.

1. **Glaucoma pulls the average down.** Most of the wrong answers come from glaucoma images being called Normal. If we look only at the other three classes, the model gets 565 out of 640 images right, which is about 88%.
2. **The VGG19 layers were frozen.** The convolutional layers keep the features they learned from ImageNet (animals, objects, scenes). They were never adjusted for eye images, so very fine signs that are specific to the retina are not picked up well.
3. **The glaucoma sign is very small.** Glaucoma mainly changes the cup inside the optic disc, which takes up only a small part of the picture. The rest of the image looks healthy, so the image as a whole looks close to a normal eye.
4. **Detail lost while resizing.** The 512×512 images are reduced to 224×224, so a lot of fine detail such as thin vessels and tiny spots is lost before the model sees the

image.

5. **Short training.** The model was trained for only 6 epochs. This was enough for DR and cataract, but not for learning the harder difference between glaucoma and normal.
6. **No data augmentation.** The training images were used as they are, without flips, rotations or brightness changes, so the model saw less variety.
7. **Different input scaling.** VGG19 was originally trained with its own colour pre-processing (mean subtraction), while we only divided by 255. The pre-trained filters still work, but not at their best.

The number of images is not the reason, because all four classes have almost the same number of images (1007 to 1098). Most of these points are easy to fix, and they are listed in the future scope.

10.4 Summary

Table 10.1: Summary of class-wise results

Class	**F1-Score**	**Remark**
Diabetic Retinopathy	98.21%	Excellent, no DR case missed
Cataract	83.20%	Good, a few mild cases called Normal
Normal	66.09%	Over-predicted, many glaucoma images end up here
Glaucoma	39.86%	Weak, needs improvement

Chapter 11

Conclusion and Future Scope

11.1 Conclusion

In this project we built Netra.AI, a deep learning system that classifies fundus images into Cataract, Diabetic Retinopathy, Glaucoma and Normal. We used transfer learning with a frozen VGG19 model and trained only a small classification head on 3,376 images. All images were resized to 224×224 and normalized before training.

The model achieved an accuracy of 73.96% on 841 validation images. It performed very well on diabetic retinopathy (F1-score 98.21%) and well on cataract (F1-score 83.20%), but it needs improvement for glaucoma. The model is part of a complete web application that explains the result using the Gemini 3.5 Flash API, helps the user find nearby eye hospitals, and is hosted on Google Cloud Run. The project helped us understand CNNs, transfer learning, model training and evaluation, and deployment in practice.

11.2 Future Scope

Table 11.1: Future scope

Area	**Improvement**
Fine-tuning	Unfreeze the last VGG19 block and train it with a small learning rate
Pre-processing	Use VGG19's own `preprocess_input` and a smoother resizing method
Longer training	Train for more epochs with early stopping
Data augmentation	Use flipping, rotation and brightness changes to get more variety
Glaucoma detection	Crop the optic disc area and train a separate model for it
Other models	Try ResNet50, EfficientNet and Vision Transformers
Explainability	Show Grad-CAM heatmaps of the area the model looked at [9]
More data	Add more datasets and test on images from other hospitals
Mobile app	Convert the model to TensorFlow Lite for offline use
Languages	Show the explanation in Hindi, Marathi and other languages

References

[1] World Health Organization, *World Report on Vision*, Geneva, 2019.

[2] Y.-C. Tham et al., "Global prevalence of glaucoma and projections of glaucoma burden through 2040," *Ophthalmology*, vol. 121, no. 11, pp. 2081–2090, 2014.

[3] V. Gulshan et al., "Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs," *JAMA*, vol. 316, no. 22, pp. 2402–2410, 2016.

[4] D. S. W. Ting et al., "Development and validation of a deep learning system for diabetic retinopathy and related eye diseases using retinal images from multiethnic populations with diabetes," *JAMA*, vol. 318, no. 22, pp. 2211–2223, 2017.

[5] A. Krizhevsky, I. Sutskever and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 2012, pp. 1097–1105.

[6] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. ICLR*, 2015.

[7] K. He, X. Zhang, S. Ren and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[8] J. Yosinski, J. Clune, Y. Bengio and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in Neural Information Processing Systems*, 2014, pp. 3320–3328.

[9] R. R. Selvaraju et al., "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE ICCV*, 2017, pp. 618–626.

[10] Keras, "Keras Applications," `https://keras.io/api/applications/`

[11] G. V. Doddi, "Eye Diseases Classification," Kaggle, `https://www.kaggle.com/datasets/gunavenkatdoddi/eye-diseases-classification`

[12] Google, "Gemini API Documentation," `https://ai.google.dev/gemini-api/docs`

[13] Box Developer Documentation, "Google Gemini 3.5 Flash Model Card," `https://developer.box.com/guides/box-ai/ai-models/google-gemini-3-5-flash-model-card`

[14] Google Cloud, "Cloud Run Documentation," https://cloud.google.com/run/docs

[15] OpenStreetMap Wiki, "Overpass API," https://wiki.openstreetmap.org/wiki/Overpass_API

Chapter A

Project Structure

```
Mini-Project/
|-- app.py                          # Flask app, model loading, prediction, Gemini
|-- web_scraper.py                  # Doctor finder
|-- model_training.py               # VGG19 training script
|-- evaluate_model.py               # Evaluation and graphs
|-- model_evaluation_metrics.csv    # Saved metrics
|-- requirements.txt                # Python libraries
|-- Dockerfile                      # Container for Cloud Run
|-- .dockerignore
|-- .env                            # API keys (not uploaded)
|-- README.md
|-- models/
|   `-- eye_disease_model.h5        # Trained model
|-- dataset/
|   |-- cataract/
|   |-- diabetic_retinopathy/
|   |-- glaucoma/
|   `-- normal/
|-- templates/                      # index, loading, result, doctors (HTML)
`-- static/                         # styles.css, script.js, assets, favicon, uploads
```

Listing A.1: Project folder structure

Disclaimer

Netra.AI is a student project made for educational purposes only. It is not a certified medical device and its results should not be used in place of a proper diagnosis. Please consult a qualified eye specialist for any eye-related problem.